
LEARN CILIUM

Livro do aluno: Cilium BGP dentro da vm-cilium

Guia de estudo e prática

Objetivo do módulo

Ao terminar este roteiro, a pessoa estudante consegue localizar cada parte do laboratório, explicar o que o Cilium anuncia por BGP e validar um serviço LoadBalancer sem alterar o ambiente.

Edição: 1.0 · Formato: laboratório guiado e reproduzível

Sumário

Capítulo	Página
Guia de estudo e prática	3
Como usar o guia	3
Mapa do laboratório	4
Vocabulário mínimo	5
A pergunta central: cada pod anuncia /32?	6
Exercício 1 — Baseline somente leitura	6
Exercício 2 — O que o Cilium anuncia	7
Exercício 3 — VIP e caminho do tráfego	8
Apêndice do instrutor	9
Limites do que foi demonstrado	10
Próximos estudos	11
Referências públicas	11

Guia de estudo e prática

Edição: 1.0 · Formato: laboratório guiado e reproduzível · Público-alvo: estudantes iniciantes de Kubernetes, redes e Cilium

Objetivo do módulo

Ao terminar este roteiro, a pessoa estudante consegue localizar cada parte do laboratório, explicar o que o Cilium anuncia por BGP e validar um serviço LoadBalancer sem alterar o ambiente.

Este guia acompanha uma execução já preparada do laboratório. Ele usa rótulos de localização para que você saiba onde cada comando roda, separa o que foi observado do que é esperado e mantém o contexto Kubernetes explícito em cada consulta.

Como usar o guia

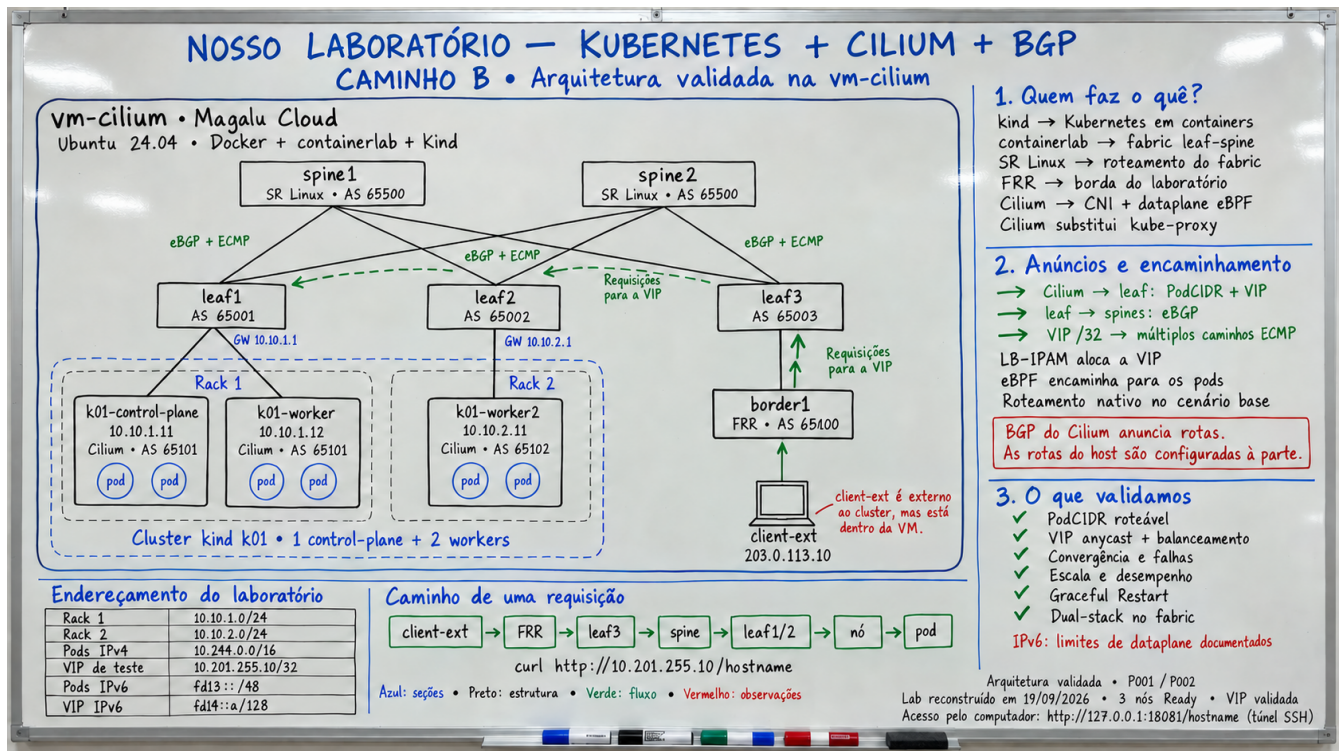
Objetivos de aprendizagem

- explicar a diferença entre uma sessão BGP, uma rota anunciada e o encaminhamento de um pacote;
- identificar onde cada comando deve ser executado;
- distinguir um PodCIDR agregado de um VIP de serviço;
- validar o estado de um cluster já ligado sem apagar pods, sessões ou containers;
- registrar separadamente a saída observada e a expectativa do roteiro.

Regras de segurança

- Os exercícios da pessoa estudante são de leitura e de requisição HTTP.
- Não use `kubectl delete`, `kubectl drain`, `docker stop`, `clab destroy` ou o script de teardown como parte deste roteiro.
- Não troque o contexto padrão do `kubectl`. Use sempre o kubeconfig dedicado e `--context kind-k01` mostrados neste guia.
- Se `127.0.0.1:18081` já responder, existe um túnel ativo. Reutilize-o e não tente abrir outro listener.
- O túnel local não publica a VIP na Internet; ele apenas encaminha uma porta do computador da pessoa estudante para a VIP dentro da VM.

Mapa do laboratório

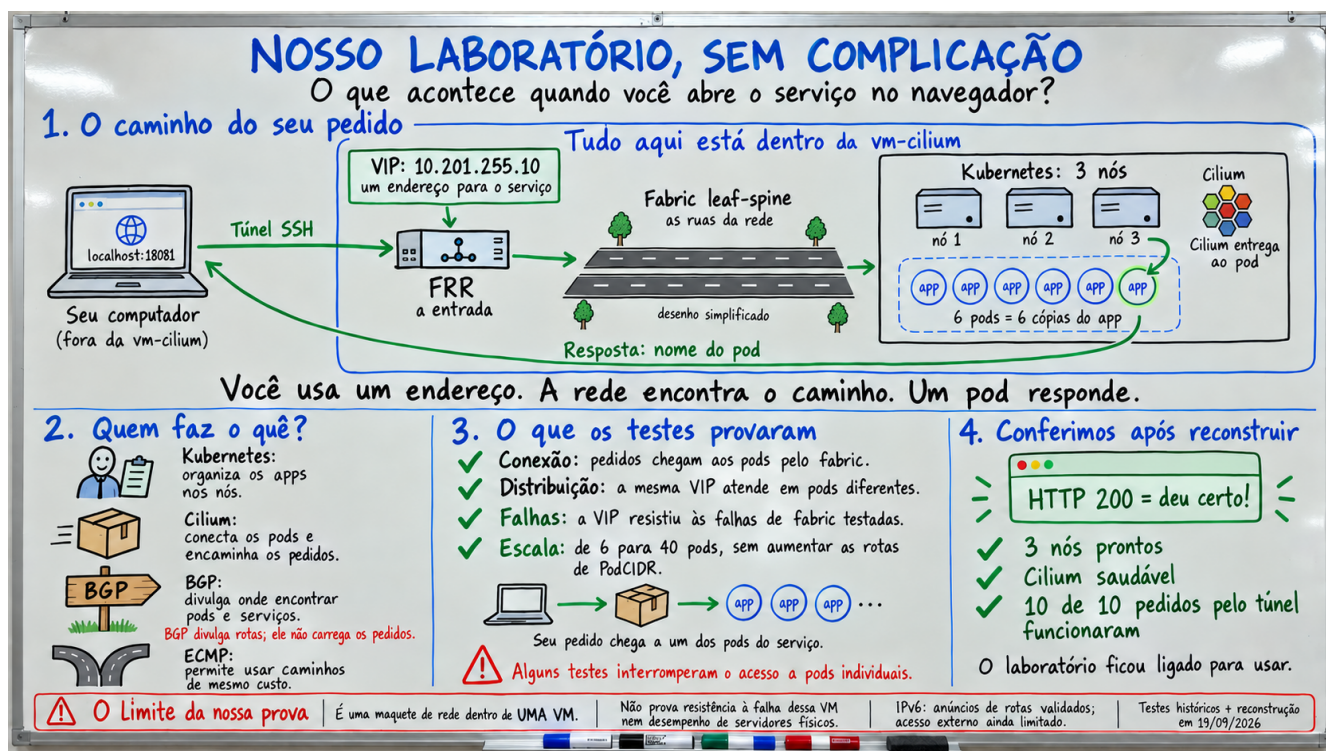


O desenho inteiro vive dentro de uma única máquina Linux chamada **vm-cilium**. O computador da pessoa estudante, chamado aqui de Mac, fica fora dela e acessa a VM por SSH. Dentro da VM existem o fabric, o roteador de borda, o cliente de teste e o cluster kind.

Rótulo	Onde fica	Papel
Mac	computador da pessoa estudante	abre SSH, túnel e <code>curl</code> local
vm-cilium	host Linux do laboratório	executa Docker, Containerlab, kind e os scripts
spine1, spine2	containers SR Linux	recebem e propagam rotas do fabric
leaf1, leaf2, leaf3	containers SR Linux	conectam racks, border e nós Kubernetes
border1	container FRR 8.4.1	borda do fabric e caminho para <code>client-ext</code>
client-ext	container dentro da VM	cliente externo ao cluster, usado para os curls
k01-control-plane, k01-worker, k01-worker2	containers kind dentro da VM	três nós Kubernetes do cluster <code>k01</code>
pods echo-*	dentro dos nós kind	seis réplicas do serviço de teste

O estado de referência deste guia é: Kubernetes **v1.35.0**, três nós, Cilium **1.20.1**, cinco nós SR Linux **25.3.2** (dois spines e três leaves), `border1` com FRR **8.4.1** e seis réplicas `echo`. São dez containers principais no conjunto: cinco SR Linux, `border1`, `client-ext` e três nós kind.

Os enlaces spine-leaf usam endereçamento IPv4 numerado em `/31`. Um texto antigo do kit usava a palavra “unnumbered”; para este laboratório, considere os `/31` reais da configuração implantada. Os nós Kubernetes entram em sub-redes de rack e anunciam seus PodCIDRs pelo Cilium.



Onde um comando deve rodar

O prompt ou a tabela do exercício informa o local. **Mac** significa o computador da pessoa estudante. **vm-cilium** significa uma sessão SSH na VM. **container** significa um comando precedido por `sudo docker exec` na VM. **Kubernetes** significa um comando precedido por `sudo KUBECONFIG=... kubectl` na VM.

Não confunda **client-ext** com uma VM externa: ele é externo ao cluster Kubernetes, mas está dentro da **vm-cilium**.

Vocabulário mínimo

Termo	Significado neste laboratório
BGP	protocolo que troca informações de alcance entre vizinhos; aqui é usado para anunciar prefixos
speaker	o agente Cilium em um nó, que mantém a sessão BGP daquele nó
PodCIDR	bloco de IP reservado para pods de um nó; neste lab é um /24 IPv4 por nó
VIP	endereço de serviço; 10.201.255.10 é o VIP IPv4 do <code>echo-anycast</code>
/24	256 endereços, usados como bloco agregado de pods de um nó
/32	um único endereço IPv4; o VIP é anunciado como /32
RIB	tabela de rotas aprendidas ou calculadas pelo protocolo
FIB	tabela efetivamente usada para encaminhar pacotes
ECMP	uso de múltiplos próximos saltos equivalentes para o mesmo prefixo
<code>externalTrafficPolicy: Cluster</code>	permite que qualquer nó anuncie o VIP e encaminhe para backends do cluster
<code>client-ext</code>	cliente de teste fora do cluster, mas dentro da VM

A pergunta central: cada pod anuncia /32?

Não neste laboratório. Uma sessão BGP não é aberta por pod. O processo Cilium em cada nó é o speaker e mantém a sessão com o leaf do rack. Para a família IPv4, o anúncio de pods é o PodCIDR /24 do nó. Se um nó recebe 10.244.1.0/24, um pod com 10.244.1.27 está coberto por esse prefixo; não nasce uma sessão nem um anúncio BGP independente para 10.244.1.27/32.

O serviço tem uma regra diferente. O echo-anycast recebe a VIP 10.201.255.10, usa `externalTrafficPolicy: Cluster` e é elegível para o anúncio de serviço. Os três nós anunciam a mesma 10.201.255.10/32; o fabric pode instalar múltiplos próximos saltos e fazer ECMP. A rota representa o serviço e seus pontos de entrada, não cada backend.

Isso explica uma observação importante da escala: quando o laboratório passou de seis para quarenta pods, os PodCIDRs continuaram agregados. A quantidade de pods não virou automaticamente uma quantidade igual de rotas BGP. O fabric aprende rotas; ele não executa HTTP, não decide se a aplicação está pronta e não instala no kernel dos nós as rotas que aprendeu. O encaminhamento final é responsabilidade do datapath do Cilium, do serviço e das tabelas de encaminhamento.

Uma alternativa para estudar depois

É possível desenhar um experimento com blocos alocados em /32 para obter seleção mais fina de prefixos. Isso ainda não significa “um pod fazendo BGP”: o Cilium continuaria sendo o speaker e anunciaria blocos alocados ao nó. Um bloco reservado também pode sobreviver à ausência momentânea de um pod, portanto não se deve presumir retirada automática no instante em que um pod morre.

O /24 já cobre um pod novo assim que ele recebe um IP do bloco do nó. Um /32 poderia permitir políticas de caminho ou retirada seletiva por endereço, mas acrescenta uma dependência de alocação, propagação e convergência para cada bloco. Recriar um pod efêmero pode dar a ele outro IP e outro UID; uma rota /32 não preserva uma sessão TCP por si só. A comparação deve ser feita em um laboratório separado, medindo tempo de anúncio, retirada e alcance direto, sem migrar o ambiente vivo. Veja o plano de estudo em `docs/proximos-estudos.md`.

Exercício 1 — Baseline somente leitura

1.1 Entrar na VM

No Mac, substitua `IP_DA_VM` pelo endereço fornecido para a sua turma:

```
export VM_CILIUM_HOST="${VM_CILIUM_HOST:-IP_DA_VM}"
ssh -i ~/.ssh/id_rsa -o IdentityAgent=SSH_AUTH_SOCK ubuntu@"${VM_CILIUM_HOST}"
```

Se a sua chave ou usuário forem diferentes, ajuste apenas esses dois valores. O override `IdentityAgent=SSH_AUTH_SOCK` faz o cliente usar o agente disponível no shell; ele não modifica o SSH da VM.

1.2 Conferir os containers

Na vm-cilium, confirme os containers do laboratório:

```
export LAB_DIR="${LAB_DIR:-/opt/poc-k8s-fabric}"
cd "${LAB_DIR}"
sudo docker ps --format 'table {{.Names}}\t{{.Status}}'
```

Você deve encontrar `spine1`, `spine2`, `leaf1`, `leaf2`, `leaf3`, `border1`, `client-ext` e os três containers `k01-*`. Se o laboratório não estiver no ar, pare aqui e avise o instrutor; não faça um redeploy como exercício da pessoa estudante.

O `client-ext` precisa conservar seu endereço IPv4 de teste. Ainda na vm-cilium:

```
sudo docker exec clab-poc-k8s-kind-client-ext ip -4 addr show dev eth1
```

Saída esperada: o endereço 203.0.113.10 aparece uma vez. Esse endereço é da rede de teste interna do laboratório.

1.3 Conferir o cluster com contexto explícito

Na vm-cilium, use o kubeconfig dedicado da reconstrução:

```
sudo KUBECONFIG=/root/.kube/k01-rebuild.config \
  kubectl --context kind-k01 get nodes -o wide

sudo KUBECONFIG=/root/.kube/k01-rebuild.config \
  kubectl --context kind-k01 get pods -A
```

Resultado esperado: os três nós `k01-*` estão **Ready**, e os pods `echo-*` estão **Running**. O snapshot público de resultados registra seis réplicas e três sessões BGP estabelecidas; a sua saída atual é a autoridade para o momento da aula.

1.4 Registrar observado versus esperado

Preencha uma linha antes de seguir:

Item	Esperado	Observado
Nós Kubernetes	3 Ready	
Réplicas echo	6/6 prontas	
Containers do lab	10 principais	
Contexto usado	kind-k01	

Se a saída divergir, descreva o desvio. Não “corrija” apagando recursos.

Exercício 2 — O que o Cilium anuncia

2.1 Ver os PodCIDsRs

Na vm-cilium:

```
sudo KUBECONFIG=/root/.kube/k01-rebuild.config \
  kubectl --context kind-k01 get nodes \
  -o custom-columns=NODE:.metadata.name,PODCIDR:.spec.podCIDR
```

Compare cada bloco com os IPs dos pods:

```
sudo KUBECONFIG=/root/.kube/k01-rebuild.config \
  kubectl --context kind-k01 get pods -o wide
```

A pergunta para responder no caderno é: “qual rota agregada cobre o IP deste pod?”. A resposta deve apontar para o /24 do nó, não para um anúncio /32 criado pelo pod.

2.2 Ver as sessões BGP do Cilium

Na vm-cilium:

```
sudo KUBECONFIG=/root/.kube/k01-rebuild.config \
  cilium --context kind-k01 bgp peers
```

Esse comando consulta os agents do cluster. Um `kubectl exec ds/cilium` escolheria apenas um pod e poderia mostrar somente a sessão daquele nó. Procure três sessões **established**, uma para cada nó; não existe um daemon BGP em cada pod `echo-*`.

Para ver os recursos declarativos:

```
sudo KUBECONFIG=/root/.kube/k01-rebuild.config \
  kubectl --context kind-k01 get ciliumbgpclusterconfigs,ciliumbgppeers,ciliumbgpadvertisements
```

2.3 Ver anúncios IPv4

Na vm-cilium:

```
sudo KUBECONFIG=/root/.kube/k01-rebuild.config \
cilium --context kind-k01 bgp routes advertised ipv4 unicast
```

Procure duas classes de prefixo:

- o PodCIDR /24 do nó que respondeu ao comando;
- a VIP de serviço, quando o anúncio de `Service` estiver ativo.

O conjunto exato de prefixos recebidos pode variar com o momento do rollout. O que não muda neste desenho é a unidade conceitual: nó anuncia PodCIDR; serviço anuncia VIP.

2.4 Confirmar a rota no spine

Na vm-cilium, consulte o `spine1`:

```
sudo docker exec clab-poc-k8s-kind-spine1 sr_cli \
"show network-instance default route-table ipv4-unicast prefix 10.244.0.0/16 longer"

sudo docker exec clab-poc-k8s-kind-spine1 sr_cli \
"show network-instance default route-table ipv4-unicast prefix 10.201.255.10/32 detail"
```

O primeiro comando mostra os blocos de pods aprendidos. O segundo mostra a VIP específica. Em uma rota /32 de VIP, vários próximos saltos podem aparecer; isso é a matéria-prima do ECMP.

Exercício 3 — VIP e caminho do tráfego

3.1 Conferir a VIP no Kubernetes

Na vm-cilium:

```
sudo KUBECONFIG=/root/.kube/k01-rebuild.config \
kubectl --context kind-k01 get svc echo-anycast echo-local -o wide
```

O `echo-anycast` deve mostrar `10.201.255.10` e o `echo-local`, quando presente, `10.201.255.11`. O primeiro usa `externalTrafficPolicy: Cluster` e pode ser anunciado por todos os três nós. O segundo usa `Local` e depende de backends locais.

3.2 Fazer requisições a partir do client-ext

Na vm-cilium:

```
for i in $(seq 1 10); do
  sudo docker exec clab-poc-k8s-kind-client-ext \
    curl -fsS --max-time 5 http://10.201.255.10/hostname
done
```

Cada linha é o `hostname` de um backend. Dez amostras não garantem round-robin, nem garantem que todos os backends apareçam: ECMP e Maglev usam hashing. No túnel SSH de referência, foram observadas 10/10 respostas HTTP 200 e cinco pods distintos; registre o que a sua execução mostrar. Não atribua essa contagem automaticamente ao loop executado no `client-ext`.

3.3 Abrir o túnel para o Mac

No Mac, primeiro teste se já existe um túnel:

```
export VM_CILIUM_HOST="{VM_CILIUM_HOST:-IP_DA_VM}"
curl --silent --fail --max-time 2 http://127.0.0.1:18081/hostname
```

Se esse comando responder, use a porta existente. Se falhar e você tiver autorização do instrutor para abrir o acesso, execute em primeiro plano:

```
ssh -i ~/.ssh/id_rsa -o IdentityAgent=SSH_AUTH_SOCK \  
-o ExitOnForwardFailure=yes -N \  
-L 127.0.0.1:18081:10.201.255.10:80 \  
ubuntu@"${VM_CILIUM_HOST}"
```

Em outro terminal do Mac:

```
curl -i http://127.0.0.1:18081/hostname
```

O navegador vê `localhost`; o destino do encaminhamento é a VIP dentro da `vm-cilium`. Isso não torna `10.201.255.10` uma URL pública.

3.4 Explicar o caminho

Complete a sequência:

```
Mac → túnel SSH → vm-cilium → clab-ext → border1/FRR → leaf3  
→ spine → leaf do rack → nó Kubernetes → Cilium → Service → pod
```

BGP participa do anúncio da rota. A requisição HTTP só acontece depois que as tabelas de encaminhamento, o Cilium e o Service entregam o pacote ao backend.

Apêndice do instrutor

O laboratório já deve estar rodando para a aula. A reconstrução abaixo é uma referência de instrutor e não é um exercício obrigatório. Ela pode recriar o cluster kind e os containers; portanto, deve ser executada somente quando o responsável confirmar que o ambiente está vazio e que o estado atual pode ser substituído.

Rebuild em dois terminais

Em uma VM Ubuntu dedicada ainda sem o kit, clone o repositório em um diretório novo e prepare as ferramentas uma vez. Não use `git clone` sobre um diretório existente:

```
sudo apt-get update  
sudo apt-get install -y git ca-certificates  
sudo git clone https://github.com/reinaldosaraiva/learn-cilium.git /opt/learn-cilium  
sudo ln -s /opt/learn-cilium/poc-k8s-fabric /opt/poc-k8s-fabric  
export LAB_DIR=/opt/learn-cilium/poc-k8s-fabric  
cd "${LAB_DIR}"  
sudo ./scripts/00-prep-vm.sh
```

Se o instrutor já instalou o kit em `/opt/poc-k8s-fabric`, use `export LAB_DIR=/opt/poc-k8s-fabric` e não repita a preparação.

Antes do deploy, habilite o encaminhamento IPv4 no host da `vm-cilium`:

```
sudo sysctl -w net.ipv4.ip_forward=1
```

Antes de começar, na `vm-cilium`, confirme que não existe uma execução que será sobrescrita:

```
sudo docker ps -a --format 'table {{.Names}}\t{{.Status}}'  
sudo kind get clusters
```

Se houver containers `clab-poc-k8s-kind-*` ou o cluster `k01`, pare e peça ao instrutor uma decisão. Não apague nem substitua o laboratório automaticamente.

Quando o responsável autorizar um rebuild em ambiente vazio, use dois terminais. No Terminal 1, o deploy da topologia aguarda o control-plane kind ficar **Ready** antes de retornar:

```
export LAB_DIR="${LAB_DIR:-/opt/poc-k8s-fabric}"
cd "${LAB_DIR}"
sudo TOPO=topo/poc-kind.clab.yml ./scripts/01-deploy-fabric.sh
```

No Terminal 2, assim que a API do kind existir, exporte o kubeconfig dedicado como root e instale o Cilium. As variáveis aparecem depois de sudo para que o script receba os valores corretos:

```
export LAB_DIR="${LAB_DIR:-/opt/poc-k8s-fabric}"
cd "${LAB_DIR}"
sudo kind export kubeconfig --name k01 \
  --kubeconfig /root/.kube/k01-rebuild.config

sudo KUBECONFIG=/root/.kube/k01-rebuild.config \
  VALUES=k8s/cilium/values-dualstack.yaml \
  ./scripts/02-install-cilium.sh
```

Depois que os agents estiverem prontos, confira o ajuste de interface. Os values atuais já definem `devices=eth+`; só faça uma correção condicional se a configuração realmente estiver diferente:

```
DEVICES=$(sudo KUBECONFIG=/root/.kube/k01-rebuild.config \
  kubectl --context kind-k01 -n kube-system get cm cilium-config \
  -o jsonpath='{.data.devices}')
if [ "${DEVICES}" != "eth+" ]; then
  sudo KUBECONFIG=/root/.kube/k01-rebuild.config \
    kubectl --context kind-k01 -n kube-system patch cm cilium-config \
    --type merge -p '{"data":{"devices":"eth+"}}'
  sudo KUBECONFIG=/root/.kube/k01-rebuild.config \
    kubectl --context kind-k01 -n kube-system rollout restart ds/cilium
  sudo KUBECONFIG=/root/.kube/k01-rebuild.config \
    kubectl --context kind-k01 -n kube-system rollout status ds/cilium --timeout=5m
fi

sudo -E KUBECONFIG=/root/.kube/k01-rebuild.config \
  RACK1_NODES="k01-control-plane k01-worker" \
  RACK2_NODES="k01-worker2" \
  ./scripts/03-apply-bgp.sh
```

O `devices=eth+` é necessário neste lab para que o bpf-lb atenda pela interface do fabric. Valide os três nós, os seis pods e as três sessões antes de ensinar o exercício.

Para que o exercício pelo navegador use o túnel SSH, o instrutor deve preparar a rota do host antes de entregar a aula. Faça isso somente depois de verificar que `clab-ext` não está sendo usado por outra execução:

```
ip link show clab-ext 2>/dev/null || true
sudo ./scripts/06-export-vip-na-vpc.sh
curl -fsS --max-time 5 http://10.201.255.10/hostname
```

O script cria a rota do host e imprime instruções adicionais para acesso por outra VM da VPC. Essas instruções incluem um toggle de `ip_spoofing_guard`; não execute esse toggle para usar o túnel SSH local. Não misture esse caminho opcional com a aula básica.

Não inclua teardown como tarefa obrigatória. A decisão de deixar o laboratório no ar ou desmontá-lo pertence ao responsável pelo ambiente.

Limites do que foi demonstrado

- A sessão BGP IPv4/IPv6 do Cilium foi observada; a VIP IPv6 externa continua limitada pela topologia IPv4 do cliente e não deve ser apresentada como alcance fim a fim.
- O lab demonstra comportamento de roteamento, ECMP e convergência em containers. Não é uma medição de desempenho de hardware nem uma garantia de alta disponibilidade de produção.
- O `externalTrafficPolicy: Cluster` permite caminhos para backends remotos. O número de pods que aparece em amostras pequenas depende de hashing e não prova distribuição uniforme.

-
- O estado do laboratório pode mudar após reinício ou reconstrução. Use a saída atual para a coluna “observado”.

Próximos estudos

Compare a agregação PodCIDR /24 com uma variante de alocação /32 em um laboratório separado e leia a proposta de isolamento declarativo por namespace em `docs/proximos-estudos.md`. Nenhuma dessas variantes é aplicada ao laboratório desta aula.

Referências públicas

- Resultados resumidos do laboratório
- Plano de próximos estudos
- [Cilium BGP Control Plane](#)
- [Configuração de anúncios BGP do Cilium](#)
- [Multi-Pool IPAM do Cilium](#)
- [Políticas Kubernetes do Cilium](#)